

CS 350 Milestone Three Input With Buttons Lab Guide

At this point in the course, you have experience working with two different output devices (an LED and a 16x2 LCD display) and one type of input through the serial port. In this lab, you will add a second LED and a button that can be used to change the state of your system. In addition, you will be building out a more complete state machine for this stage of your development.

Note: All column letters provided in this document are based on the left-hand side (or bottom) of the board, as the column lettering is still mostly visible.

Step 1: Installing Packages

You need to install an additional Python package to develop an enhanced state machine. Run the following command to download the package:

```
sudo pip3 install python-statemachine
```



```
jayme — jayme@cs350lab: ~ — ssh jayme@10.67.153.71 — 88x22
jayme@cs350lab:~$ sudo pip3 install python-statemachine
[sudo] password for jayme:
Collecting python-statemachine
  Downloading python_statemachine-2.3.6-py3-none-any.whl.metadata (14 kB)
  Downloading python_statemachine-2.3.6-py3-none-any.whl (41 kB)
    41.5/41.5 kB 331.8 kB/s eta 0:00:00
Installing collected packages: python-statemachine
Successfully installed python-statemachine-2.3.6
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting
behaviour with the system package manager. It is recommended to use a virtual environme
nt instead: https://pip.pypa.io/warnings/venv
jayme@cs350lab:~$
```

Step 2: Power Off Your Raspberry Pi

Your Raspberry Pi must be shut down and unplugged to continue. To safely shut down the operating system running on your Raspberry Pi, enter the following command from a terminal (or the console) of the Raspberry Pi:

```
sudo halt
```

You will have to enter your password as you are executing the halt command with root privileges by calling it with *sudo*. Please wait at least two minutes after your terminal window closes before unplugging your Raspberry Pi.

Step 3: Moving the Existing LED

So far, you have been working with the red LED in the same spot on the solderless breadboard since the Module One assignment. However, to move forward with your updates and accommodate the physical space necessary for your buttons, you must relocate your red LED. This is one of the benefits of using a

solderless breadboard because you can move the components around, and as long as you don't change where they connect on the GPIO interface, your software will not change.

Begin by removing the red LED, the associated resistor, and both jumper wires. You will reconnect the jumper wires to the same locations with respect to the GPIO breakout board, but the location of the LED will be different.

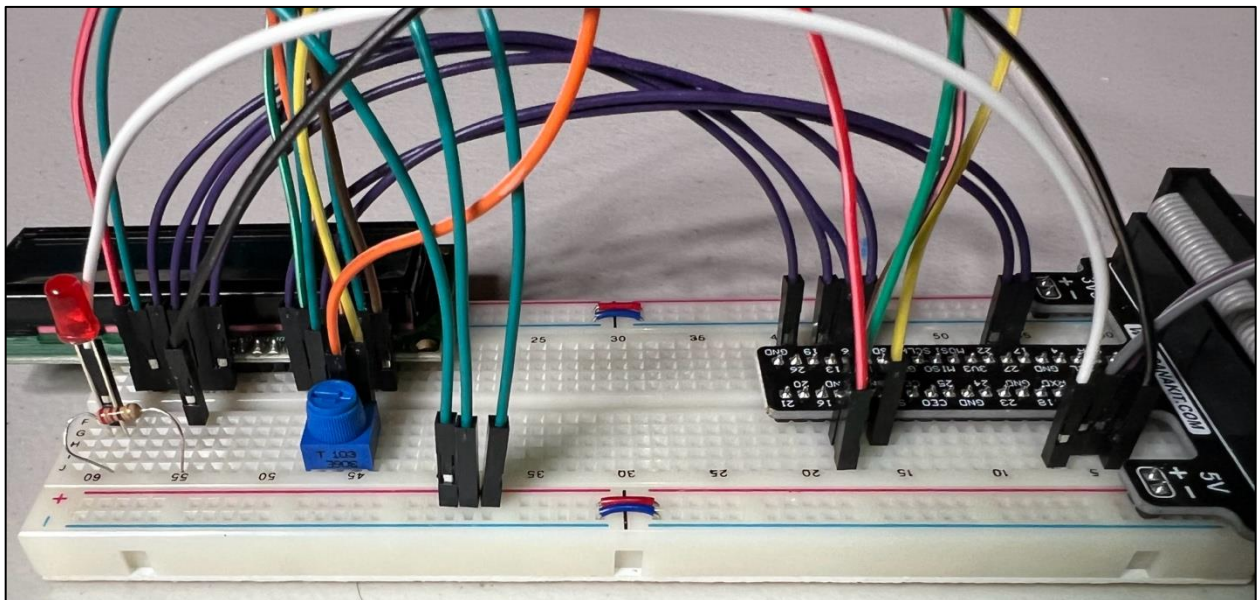
Reconnect one of the jumper wires to Column J, Row 7 (the sixth pin on the right-hand side of the GPIO breakout board when viewed as a capital T, labeled 18) and the other end to Column F, Row 60.

Reconnect one of the jumper wires to Column J, Row 4 (the third pin on the right-hand side of the GPIO breakout board when viewed as a capital T, labeled "gnd") and the other end to Column F, Row 55.

Take the red LED and connect the anode (long leg) to Column G, Row 60, and the cathode (short leg) to Column G, Row 59.

Take the 220 ohm (stripe pattern: red, red, brown) resistor, gently bend the wire legs on each side, and insert one side into Column I, Row 59, and the other into Column I, Row 55.

Note: Do not let the leg of the resistor physically contact the LED or the resistor for the red LED, as this could cause a short and damage your circuit (or your Raspberry Pi).



Now that you have relocated the red LED on your solderless breadboard, the best thing to do is to test it before moving forward. Since the light is still connected to the same GPIO line (GPIO 18), you can use the ON/OFF scripts you used in the Module One Lab to test the new configuration. These commands are:

```
sudo python3 GPIO-18-ON.py  
sudo python3 GPIO-18-OFF.py
```

Step 4: Power Off Your Raspberry Pi

Your Raspberry Pi must be shut down and unplugged to continue. To safely shut down the operating system running on your Raspberry Pi, enter the following command from a terminal (or the console) of the Raspberry Pi:

```
sudo halt
```

You will have to enter your password as you are executing the halt command with root privileges by calling it with *sudo*. Please wait for at least two minutes after your terminal window closes before unplugging your Raspberry Pi.

Step 5: Building the New Circuit (LED)

Components

- 1 x blue LED
- 1 x 220 ohm resistor (stripe pattern: red, red, brown)
- 1 x button (with your choice of keycap)
- 1 x 10K ohm resistor (stripe pattern: brown, black, orange)
- 4 x male-to-male jumper wires



Blue LED



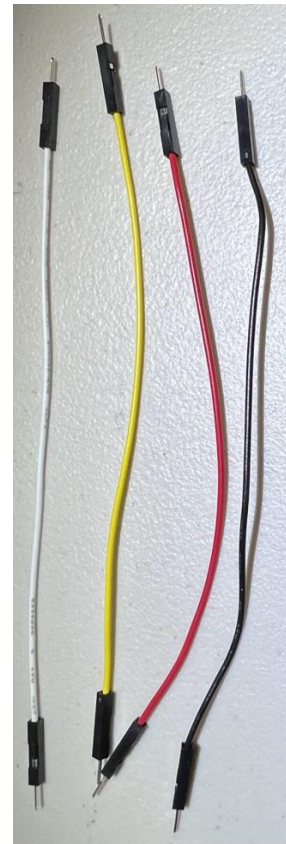
220 ohm
resistor



Button



10K ohm
resistor



Male-to-male
jumper wires

The first item that you will connect is the blue LED. This connection will be a repeat of connecting the red LED in a different location.

Remember: Of the components you are using in this step, the blue LED is specific about which way it is used in the circuit. The longer leg of the LED is called the anode (+) and needs to be connected to the GPIO pin where it will receive voltage when the GPIO pin is set to high (sometimes referred to as True) in output mode. The shorter leg of the LED is called the cathode (-) and will be connected to the resistor that will be tied to the ground pin on the GPIO header.

When installing, use pins 7 (Ground) and 8 (GPIO 23) on the right-hand side (when viewed as a T) of the GPIO breakout board.

Connect one jumper wire to Column J, Row 8 (this should be the seventh pin on the right-hand side of your breakout board and labeled "gnd") to Column J, Row 49.

Remember: The columns in this bank of pins were labeled A, B, C, D, and E from right to left at the top of the board. Use the references at the bottom of the board because they are still visible.

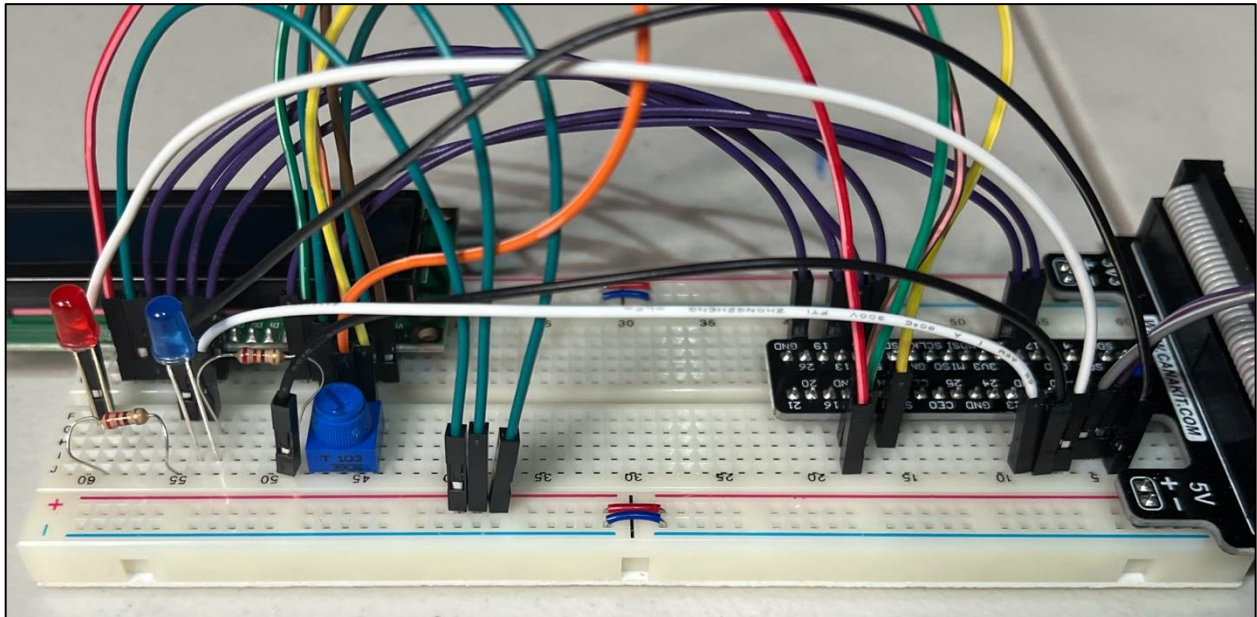
Choose one jumper wire and connect Column J, Row 9 (this should be the eighth pin on the right-hand side, labeled 23 on the breakout board) and connect it to Column F, Row 54.

Take the 220 ohm (stripe pattern: red, red, brown) resistor, gently bend the wire legs on each side, and insert one side into Column G, Row 53, and the other in Column G, Row 49.

Note: Do not let the leg of the resistor physically contact the LED or the resistor for the red LED, as this could cause a short and damage your circuit (or your Raspberry Pi).

Take the blue LED and connect the anode (long leg) to Column I, Row 54, and the cathode (short leg) to Column I, Row 53.

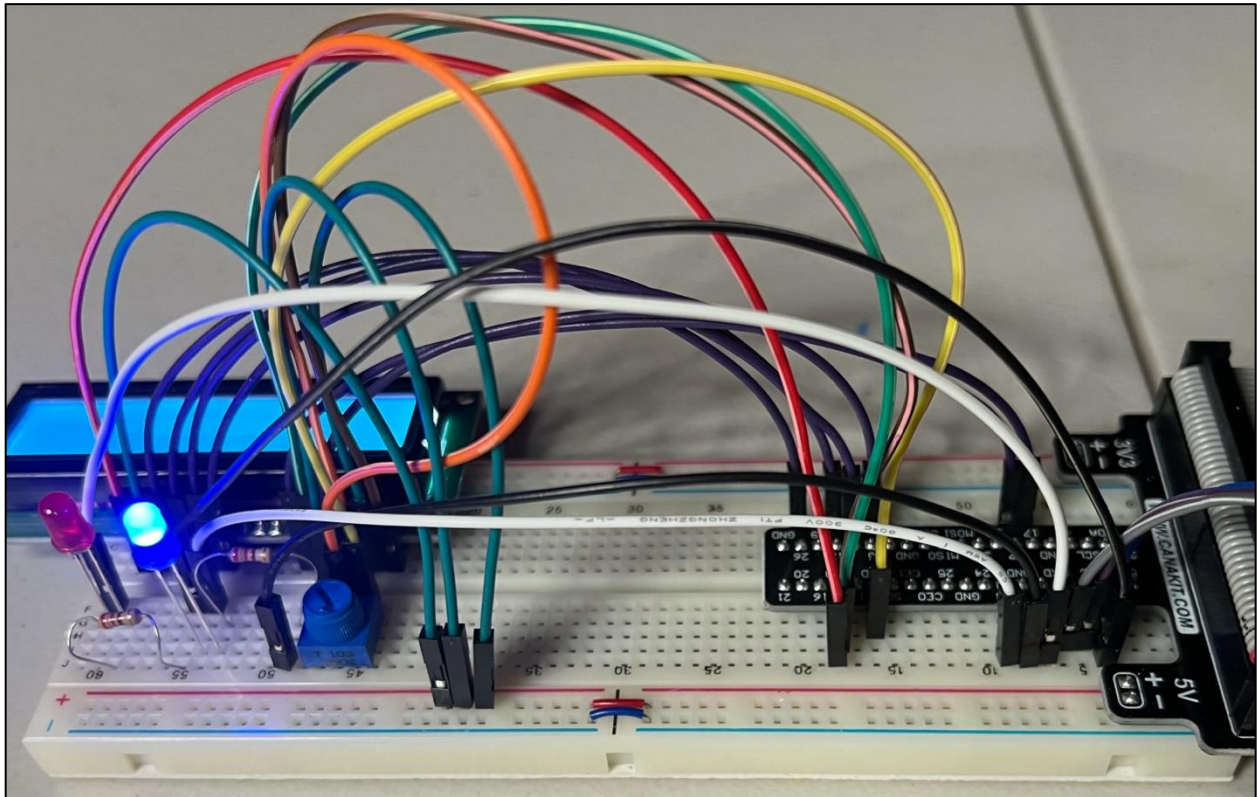
With the blue LED connected, your board should look like the following image:



Now that you have relocated the blue LED on your solderless breadboard, the best thing to do is to test it before moving forward. To accomplish this, reconnect power to your Raspberry Pi and run the following test scripts:

```
GPIO-23-ON.py  
GPIO-23-OFF.py
```

The light is connected to GPIO line 23 and should turn on and off using the individual scripts.



Step 6: Power Off Your Raspberry Pi

Your Raspberry Pi must be shut down and unplugged to continue. To safely shut down the operating system running on your Raspberry Pi, enter the following command from a terminal (or the console) of the Raspberry Pi:

```
sudo halt
```

You will have to enter your password as you are executing the halt command with root privileges by calling it with *sudo*. Please wait for at least two minutes after your terminal window closes before unplugging your Raspberry Pi.

Step 7: Building the New Circuit (Button)

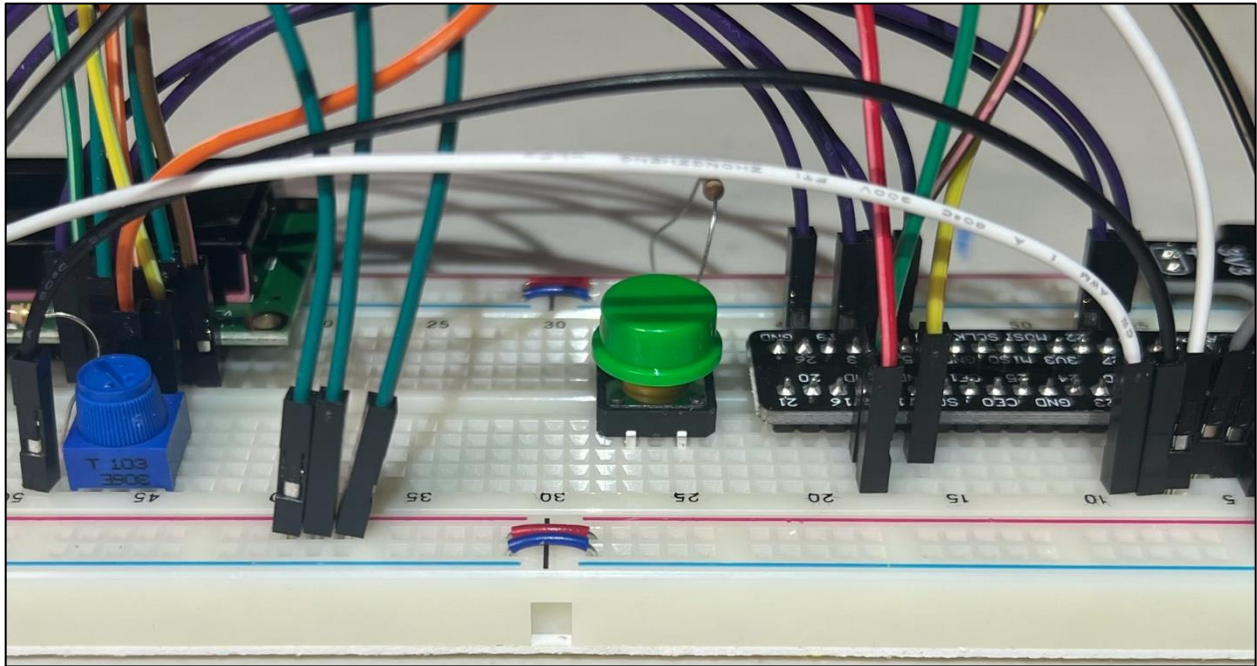
The next component you need to connect is the button, which takes up a lot of real estate on the solderless breadboard; you need to make certain that it is positioned to leave room for connecting other components in the future. Each button has four pins, and the pair of pins opposite each other are connected. Pressing the button makes momentary contact between the normally separated pins.

Note: Up to this point, you have been using the numbering on the right-hand side of the solderless breadboard (looking at the board where the GPIO breakout board looks like a capital T). However, for the buttons, you must switch to the numbering on the left-hand side of the board, as it is closer to the components.

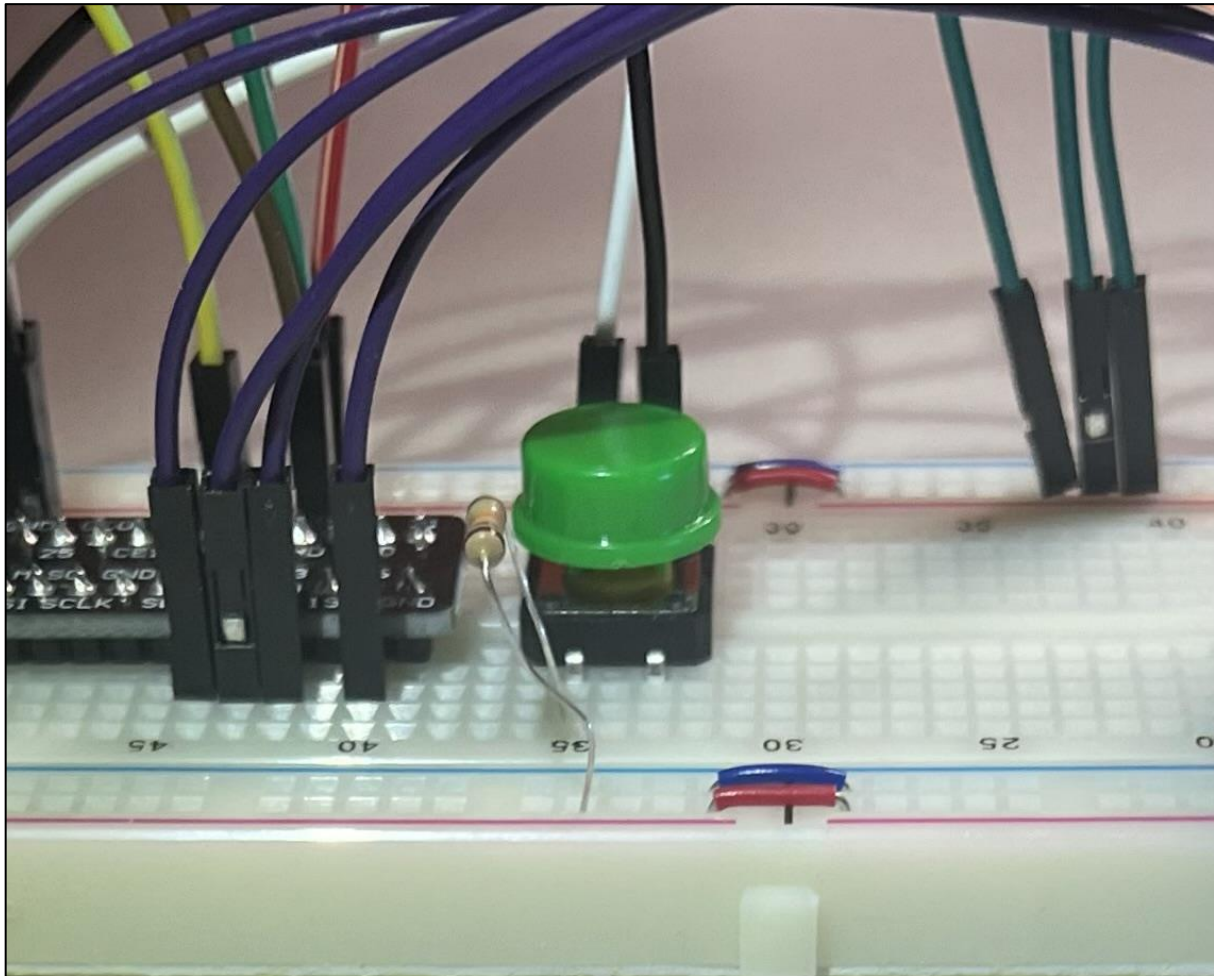
Position the button so that the pins on one side of the switch are in Column D, Rows 33 and 35, while the other side is connected to Column G, Rows 33 and 35.

Note: The button will straddle the ditch between the rows of connected pins.

Once you have the button in place and the pins in the correct holes, press firmly to seat it. This will require pressure at each of the four corners of the button. You will know when the button has been completely seated when the base of the button is in contact with the solderless breadboard and it does not rock when you press on any one side of the button.



The next component to place is the 10K ohm resistor (stripe pattern: brown, black, orange). One leg of the resistor will need to be inserted into the +3.3v power rail at Row 35 while gently bending the legs so that the other leg can be inserted in Column B, row 35.



Next, you have to connect two jumper wires.

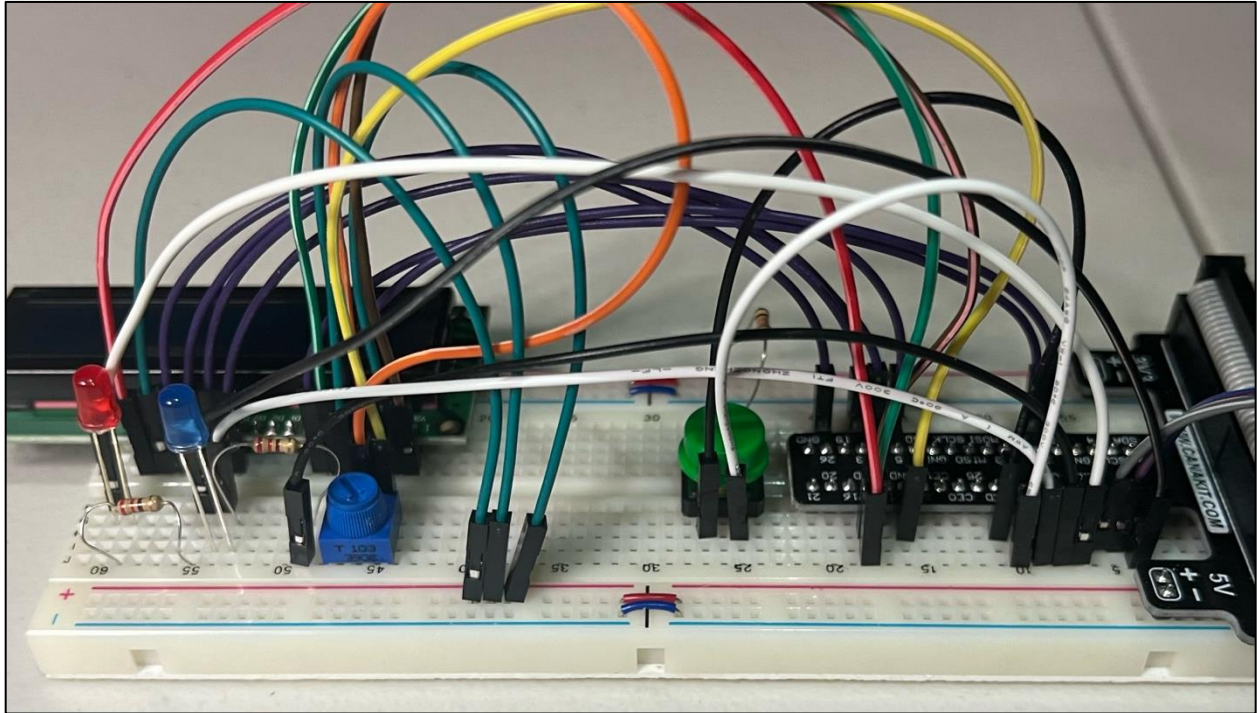
The first jumper wire you connect attaches the second half of the switch to ground. This is necessary so that when you press the button, the circuit is completed, and electricity flows from power through the resistor to ground.

Connect the next jumper wire to Column H, Row 33 (using the numbering from the left-hand side of the board as viewed when the GPIO breakout board looks like a capital T) to Column H, Row 11 (this should be the 10th pin on the right-hand side of the GPIO breakout board, labeled “gnd”).

The next jumper wire will connect the switch (on the same side as the resistor) to the GPIO pin being used to monitor the button.

The last jumper wire will connect to Column H, Row 35 (the same row as one of the pins on the switch) and connect it to Column J, Row 10 (this should be the ninth pin on the right-hand side, labeled 24 on the breakout board).

This completes your circuit setup. The image below is an example of how your solderless breadboard should look when viewed from the right-hand side of the GPIO:

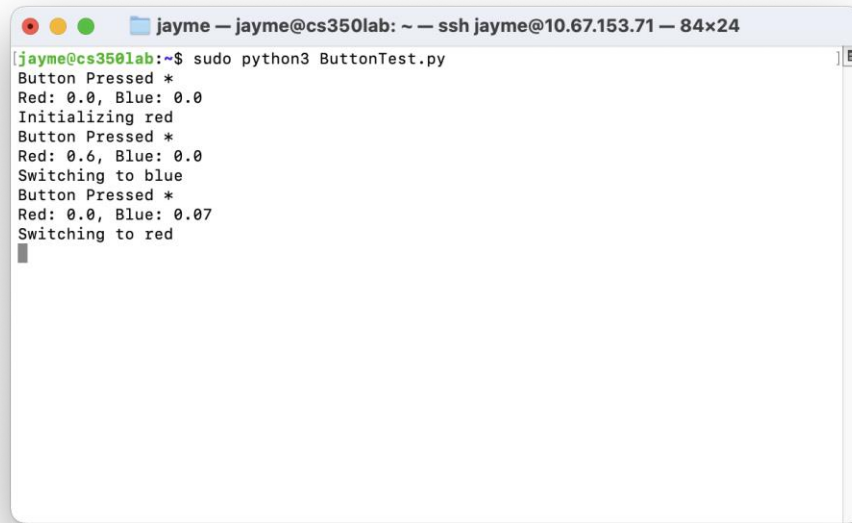


Reconnect the power to your Raspberry Pi, and log in.

Step 8: Testing Your Circuit

Now that the circuit is complete, it is time to test. The first script you will use is `ButtonTest.py`. Like the other testing scripts, this one requires elevated privileges and must be run with `sudo`.

```
sudo python3 ButtonTest.py
```



```
jayme — jayme@cs350lab: ~ — ssh jayme@10.67.153.71 — 84x24
[jayme@cs350lab:~]$ sudo python3 ButtonTest.py
Button Pressed *
Red: 0.0, Blue: 0.0
Initializing red
Button Pressed *
Red: 0.6, Blue: 0.0
Switching to blue
Button Pressed *
Red: 0.0, Blue: 0.07
Switching to red
```

By default, the ButtonTest.py script has its DEBUG flag set, so additional information will be printed to the console so that you can see what is happening during execution.

When you start the script, nothing visibly happens. Press and release the button you wired up on your solderless breadboard and validate that the red LED begins to fade in and out. Press and release the button again and verify that the red LED shuts off and the blue LED begins to fade in and out.

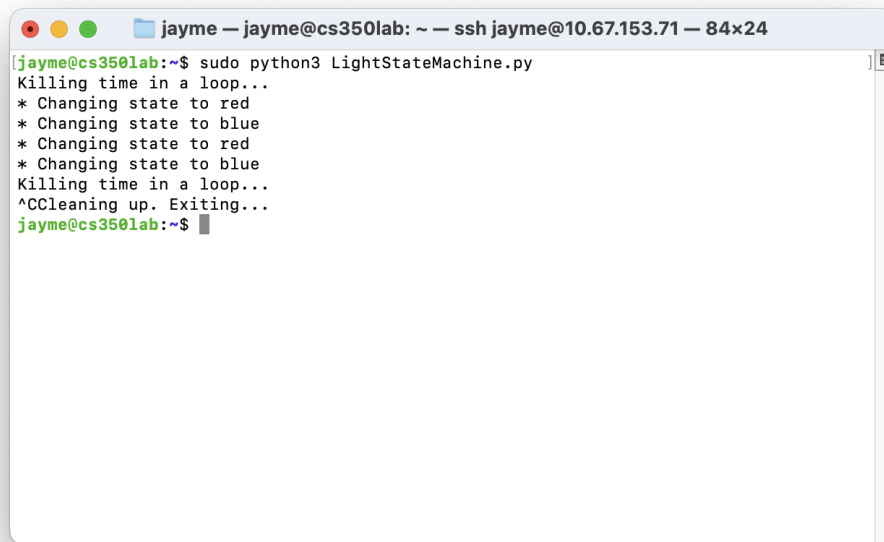
Please review the test script to understand how the button and LEDs are being manipulated.

Step 9: Building a State Machine

Once the circuit's functionality has been verified, it is time to examine an example state machine that can manage the devices on your board.

The source code for Module Five includes the LightStateMachine.py file. You can run this file similarly to the ButtonTest.py file by entering the following command:

```
sudo python3 LightStateMachine.py
```



```
jayme — jayme@cs350lab: ~ — ssh jayme@10.67.153.71 — 84x24
[jayme@cs350lab:~]$ sudo python3 LightStateMachine.py
Killing time in a loop...
* Changing state to red
* Changing state to blue
* Changing state to red
* Changing state to blue
Killing time in a loop...
^CCleaning up. Exiting...
jayme@cs350lab:~$
```

You will notice that, aside from different debug messages, the results from pressing the button should be the same as those in the ButtonTest.py script.

So, what is different, and why is this methodology for getting those results important?

This example creates a state machine complete with defined states, transitions, and events utilizing the `python-statemachine` package. This package handles all the heavy lifting of building the state machine, allowing you to concentrate on behaviors and functionality.

Step 10: Completing the Milestone

Now that you have seen the benefits of working with more complete state machines, it is time to take the next step. Using the Milestone3.py software template, address each TODO section in the code to build an application that utilizes the components you currently have in your circuit.

You will use each of the two LEDs in the circuit to flash a Morse code message. In this project, you will use the red LED to flash the dots and the blue LED to flash the dashes. You will need to specify the timing for each operation. The timings are as follows:

Object	Timing
'Dot'	500ms
'Dash'	1500ms
Pause between dots/dashes	250ms
Pause between letters	750ms
Pause between words	3000ms

Your program must also exhibit the following behavior:

1. When the button is pressed, the message being transmitted will change. **Note:** The button press should not interrupt the current message but change it for the next pass.
2. The message will repeat indefinitely until the application is stopped.
3. The application will exit gracefully, cleaning up all GPIO assets.
4. The application will communicate what is being sent using the 16x2 display.

Lab Questions

To complete this assignment, you will submit your completed Python source file, a video showing your project in action, including what happens when you press the button, and a Word document addressing the following questions:

1. Why does the loop that processes the LED blinking need to run in a separate thread?
2. What is the purpose of returning to the off state after each completed state action?
3. How could you integrate serial communications to facilitate changing the messages available to the program?
4. How could you use the 16x2 display to provide debugging information to the user when they don't have access to the application console?